MODEL DEVELOPMENT AND EVALUATION

PREPARED BY:

OBA OLAITAN - obaolaitan3@gmail.com

TEMILOLUWA MAYOWA AWOYEMI - awoyemitemi2019@gmail.com

IBEKWE GHISLIAN CHIGOZIE - chogozieghislian@gmaiL.com

In [3]:
```
# importing the necessary libraries

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import OneHotEncoder
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestRegressor
from sklearn import tree
from sklearn.tree import DecisionTreeRegressor
```

In [4]:
```
# loading the dataset

df = pd.read_csv('C:\\Users\\Hp\\Documents\\CPD\\Churn Modeling.csv')
```

In [5]:
```
df.head()

# prints out the first 5 rows of the dataset
```

Out [5]:

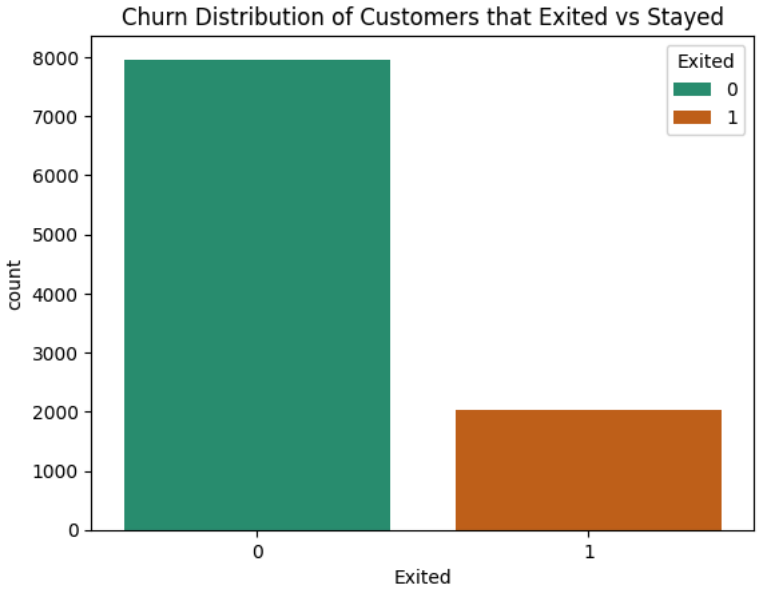
	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exite
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	1	101348.88	1
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	1	112542.58	0
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1	0	113931.57	1
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0	0	93826.63	0
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	1	79084.10	0

In [6]:
```
# Drop identifier columns that add no predictive value
df = df.drop(columns=['RowNumber', 'CustomerId', 'Surname'])
df.head()
```

Out [6]:

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
0	619	France	Female	42	2	0.00	1	1	1	101348.88	1
1	608	Spain	Female	41	1	83807.86	1	0	1	112542.58	0
2	502	France	Female	42	8	159660.80	3	1	0	113931.57	1
3	699	France	Female	39	1	0.00	2	0	0	93826.63	0
4	850	Spain	Female	43	2	125510.82	1	1	1	79084.10	0

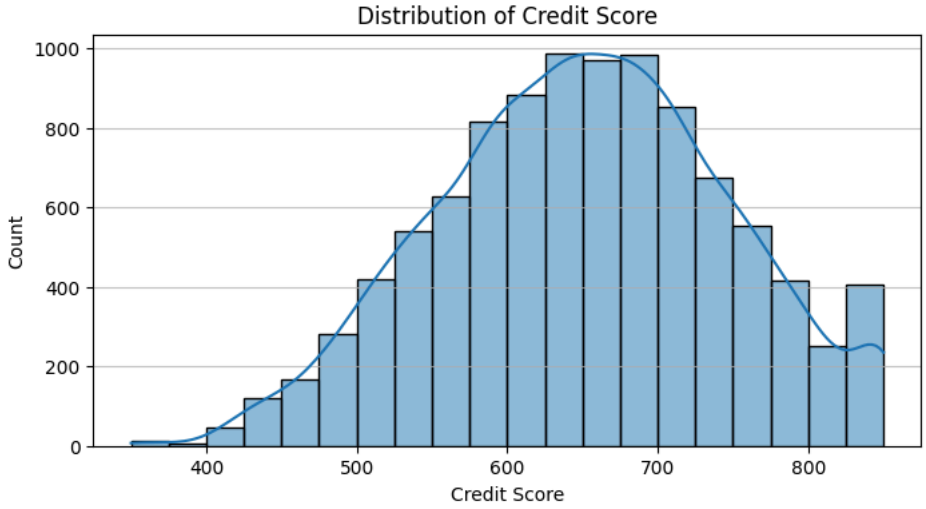
In [7]:
```
### Target Variable Exploration
# this visualizes the difference between customers who stayed and customers who exited
sns.countplot(x='Exited', data=df, hue="Exited", palette="Dark2")
plt.title('Churn Distribution of Customers that Exited vs Stayed')
plt.show()
```

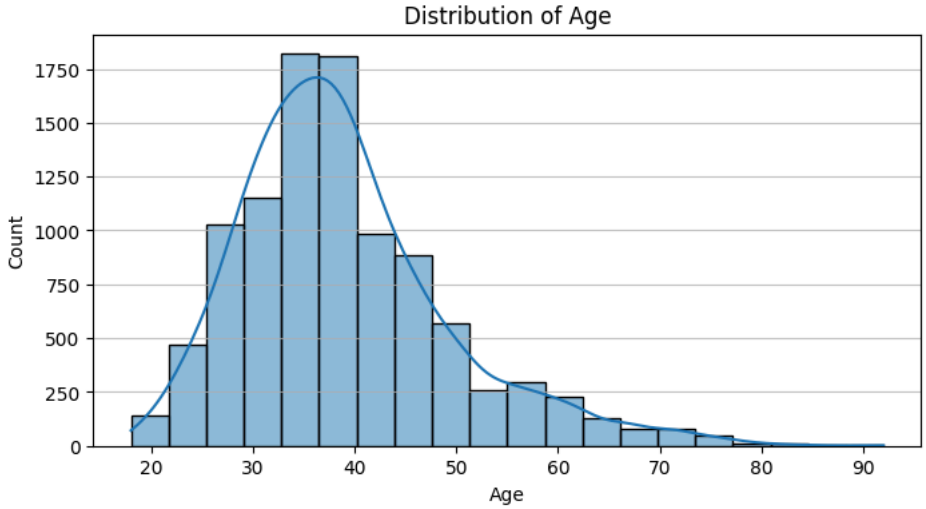


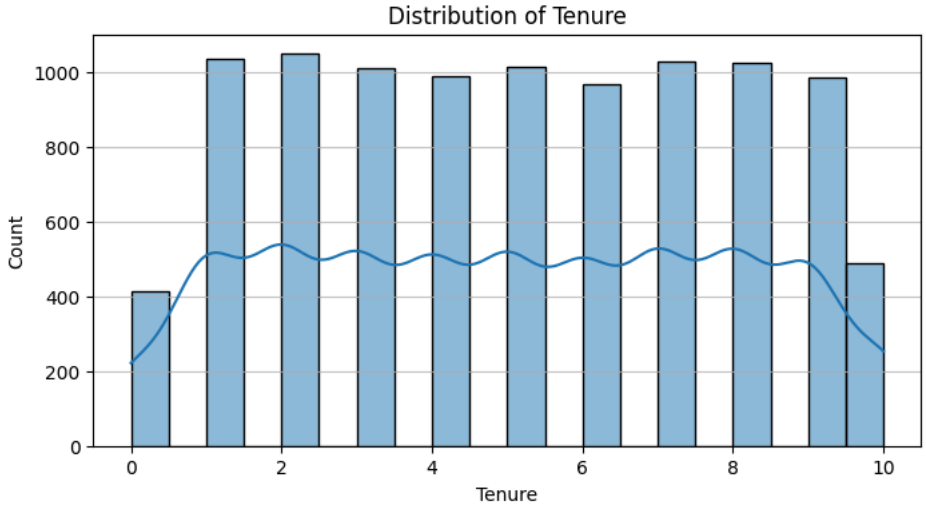
In [8]:
```
# this will give us the percentage distribution of the exited column
#as it will help us to understand the proportion of customesrs who churned and those who stayed with the company (bank)
counts = df['Exited'].value_counts(normalize=True) * 100
counts.index = ['Stayed', 'Left']
print(counts)
```

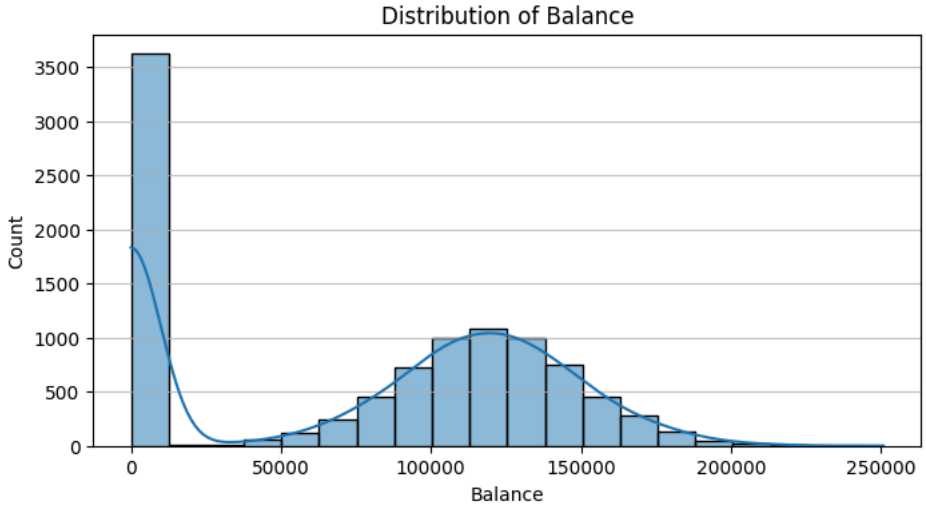
```
Stayed    79.63
Left      20.37
Name: proportion, dtype: float64
```

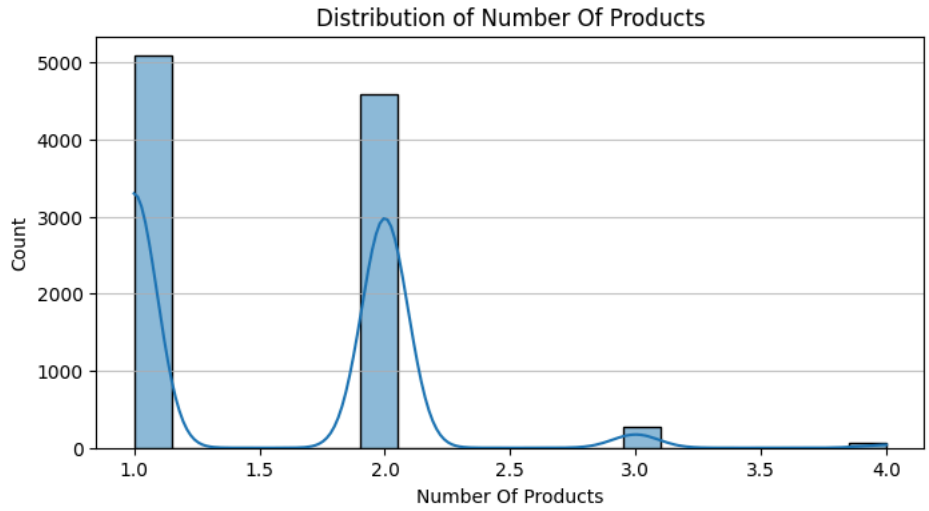
In [9]:
```
# Visualization of the distribution of the numerical features using histogram plots

plt.figure(figsize=(8, 4))
sns.histplot(data=df, x='CreditScore', kde=True, bins=20)
plt.title('Distribution of Credit Score')
plt.xlabel('Credit Score')
plt.ylabel('Count')
plt.grid(axis='y', alpha=0.75)
plt.show()

plt.figure(figsize=(8, 4))
sns.histplot(data=df, x='Age', kde=True, bins=20)
plt.title('Distribution of Age')
plt.xlabel('Age')
plt.ylabel('Count')
plt.grid(axis='y', alpha=0.75)
plt.show()

plt.figure(figsize=(8, 4))
sns.histplot(data=df, x='Tenure', kde=True, bins=20)
plt.title('Distribution of Tenure')
plt.xlabel('Tenure')
plt.ylabel('Count')
plt.grid(axis='y', alpha=0.75)
plt.show()

plt.figure(figsize=(8, 4))
sns.histplot(data=df, x='Balance', kde=True, bins=20)
plt.title('Distribution of Balance')
plt.xlabel('Balance')
plt.ylabel('Count')
plt.grid(axis='y', alpha=0.75)
plt.show()

plt.figure(figsize=(8, 4))
sns.histplot(data=df, x='NumOfProducts', kde=True, bins=20)
plt.title('Distribution of Number Of Products')
plt.xlabel('Number Of Products')
plt.ylabel('Count')
plt.grid(axis='y', alpha=0.75)
plt.show()

plt.figure(figsize=(8, 4))
sns.histplot(data=df, x='EstimatedSalary', kde=True, bins=20)
plt.title('Distribution of Estimated Salary')
plt.xlabel('Estimated Salary')
plt.ylabel('Count')
plt.grid(axis='y', alpha=0.75)
plt.show()
```

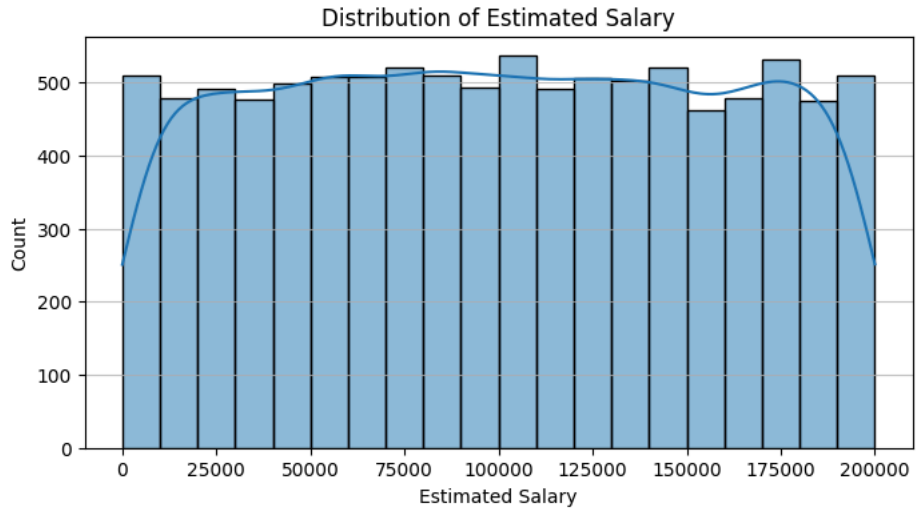

Distribution of Credit Score
Count
Credit Score
0
200
400
600
800
1000
400
500
600
700
800


Distribution of Age
Count
Age
0
250
500
750
1000
1250
1500
1750
20
30
40
50
60
70
80
90


Distribution of Tenure
Count
Tenure
0
200
400
600
800
1000
0
2
4
6
8
10


Distribution of Balance
Count
Balance
0
500
1000
1500
2000
2500
3000
3500
0
50000
100000
150000
200000
250000



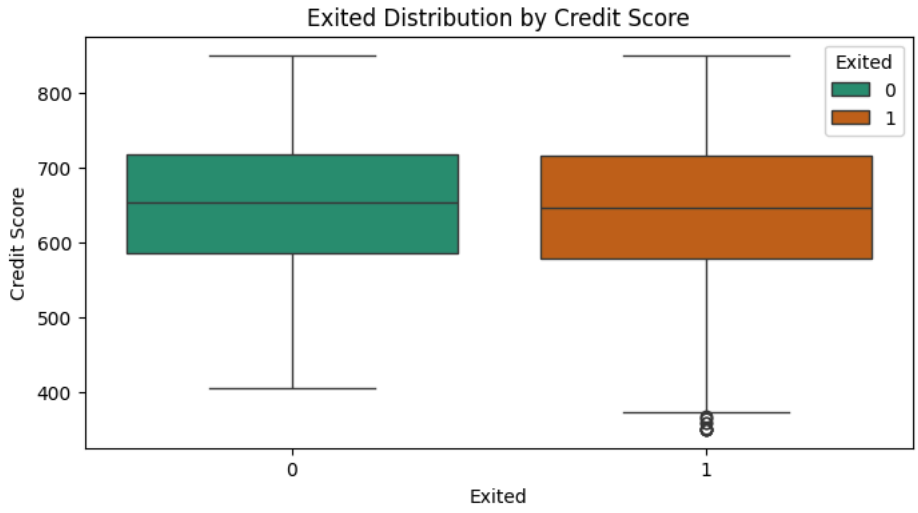


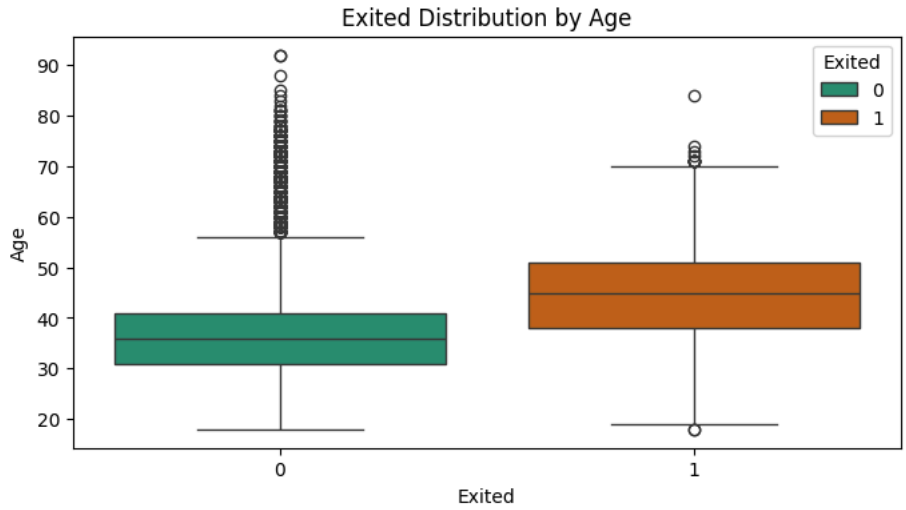
CreditScore is roughly symmetric, which is fairly typical for a large customer base. Age skews slightly right, meaning the data leans more toward middle aged customers than younger ones. Tenure is evenly spread with no strong bias toward new or long standing accounts. Balance shows a bimodal pattern, a large concentration at zero and a separate cluster of higher value accounts, which is pretty common in retail banking. NumOfProducts is heavily skewed toward 1 and 2, with very few customers holding more than that. EstimatedSalary looks almost uniformly distributed, which suggests it could be synthetic or pre-binned. Nothing here demands immediate transformation, though the zero heavy Balance column is worth keeping in mind going into modeling.

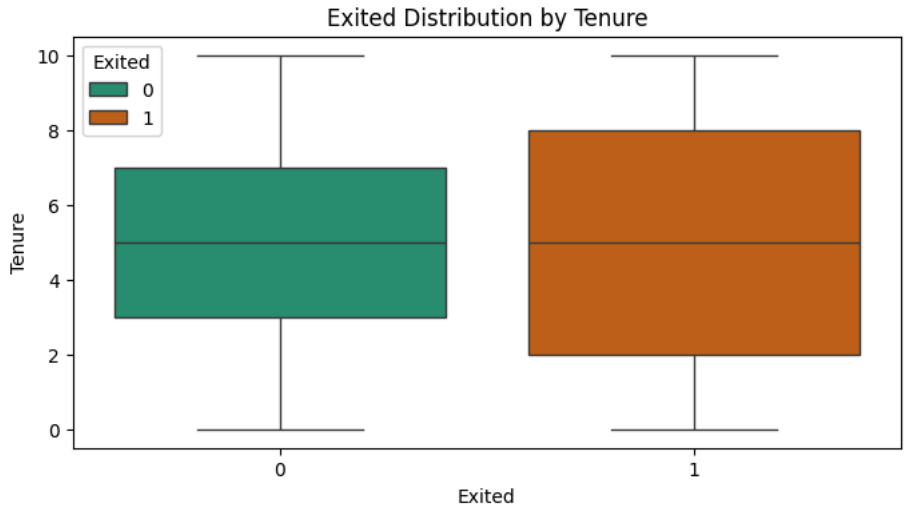
In [10]:

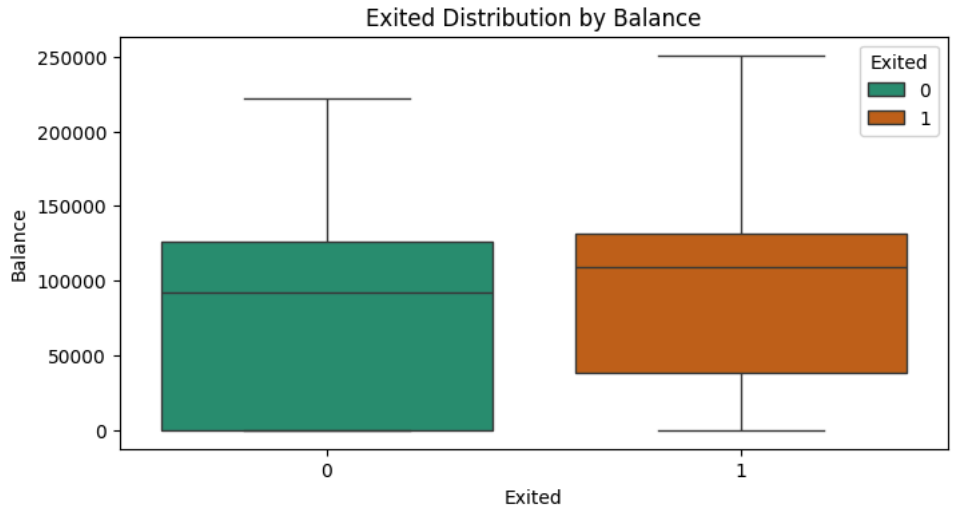
```
# Visualize the distribution of numerical features against the target variable using boxplots

# Visualization of the distribution of numerical features against the target feature using histogram plots

plt.figure(figsize=(8, 4))
sns.boxplot(data=df, x='Exited', y='CreditScore', hue='Exited', palette="Dark2")
plt.title("Exited Distribution by Credit Score")
plt.xlabel("Exited")
plt.ylabel("Credit Score")
plt.show()

plt.figure(figsize=(8, 4))
sns.boxplot(data=df, x='Exited', y='Age', hue='Exited', palette="Dark2")
plt.title("Exited Distribution by Age")
plt.xlabel("Exited")
plt.ylabel("Age")
plt.show()

plt.figure(figsize=(8, 4))
sns.boxplot(data=df, x='Exited', y='Tenure', hue='Exited', palette="Dark2")
plt.title("Exited Distribution by Tenure")
plt.xlabel("Exited")
plt.ylabel("Tenure")
plt.show()

plt.figure(figsize=(8, 4))
sns.boxplot(data=df, x='Exited', y='Balance', hue='Exited', palette="Dark2")
plt.title("Exited Distribution by Balance")
plt.xlabel("Exited")
plt.ylabel("Balance")
plt.show()

plt.figure(figsize=(8, 4))
sns.boxplot(data=df, x='Exited', y='NumOfProducts', hue='Exited', palette="Dark2")
plt.title("Exited Distribution by NumOfProducts")
plt.xlabel("Exited")
plt.ylabel("NumOfProducts")
plt.show()

plt.figure(figsize=(8, 4))
sns.boxplot(data=df, x='Exited', y='EstimatedSalary', hue='Exited', palette="Dark2")
plt.title("Exited Distribution by Estimated Salary")
```

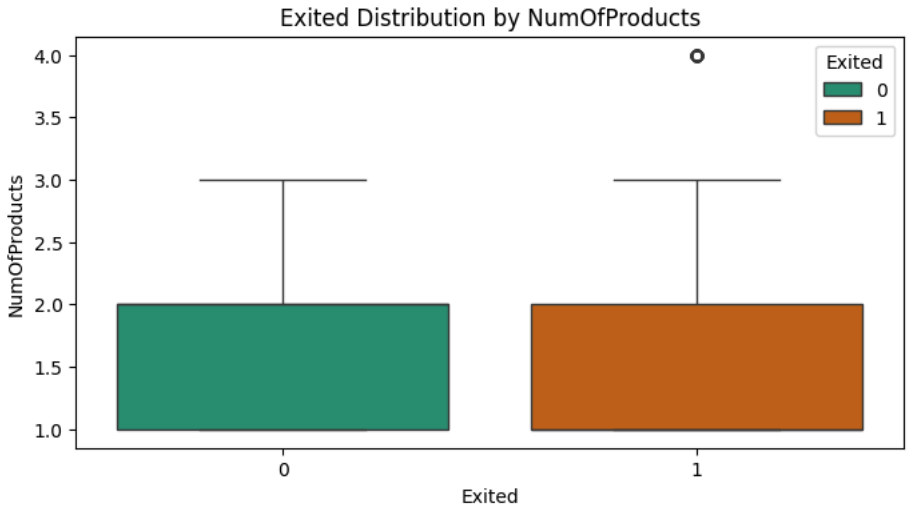
```
plt.xlabel("Exited")
plt.ylabel("Estimated Salary")
plt.show()
```

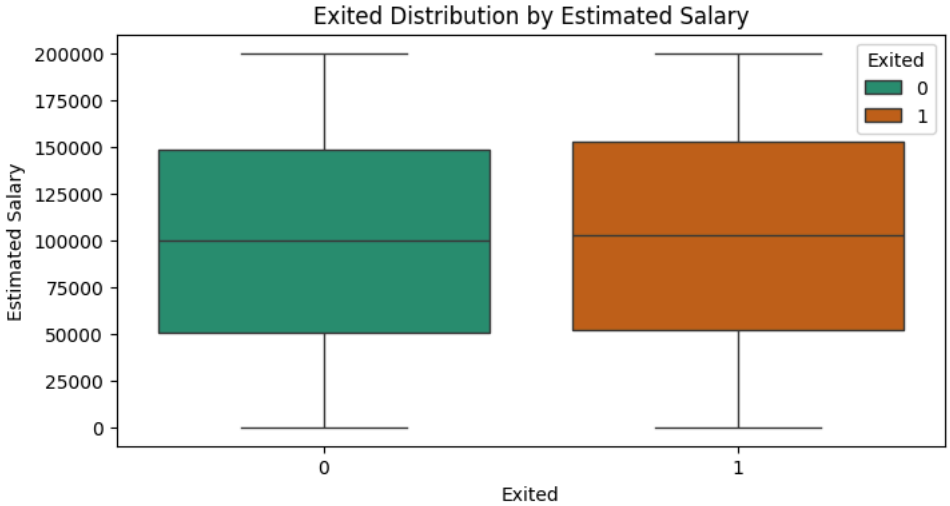






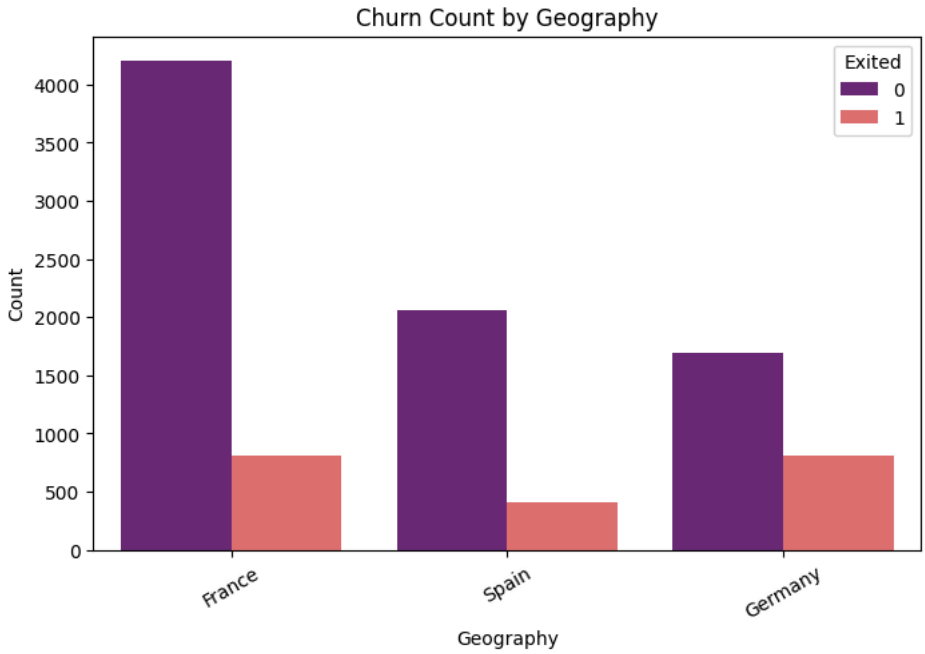


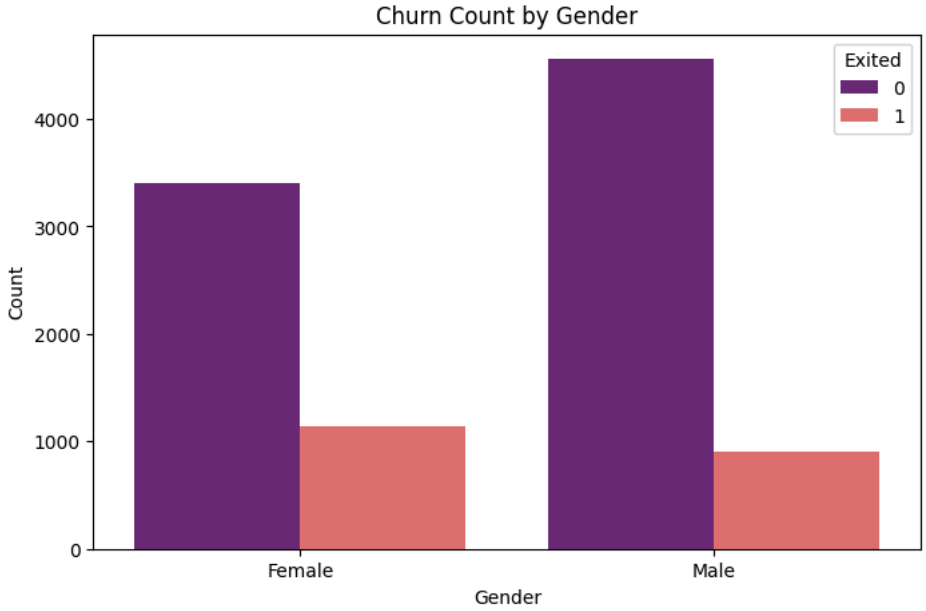


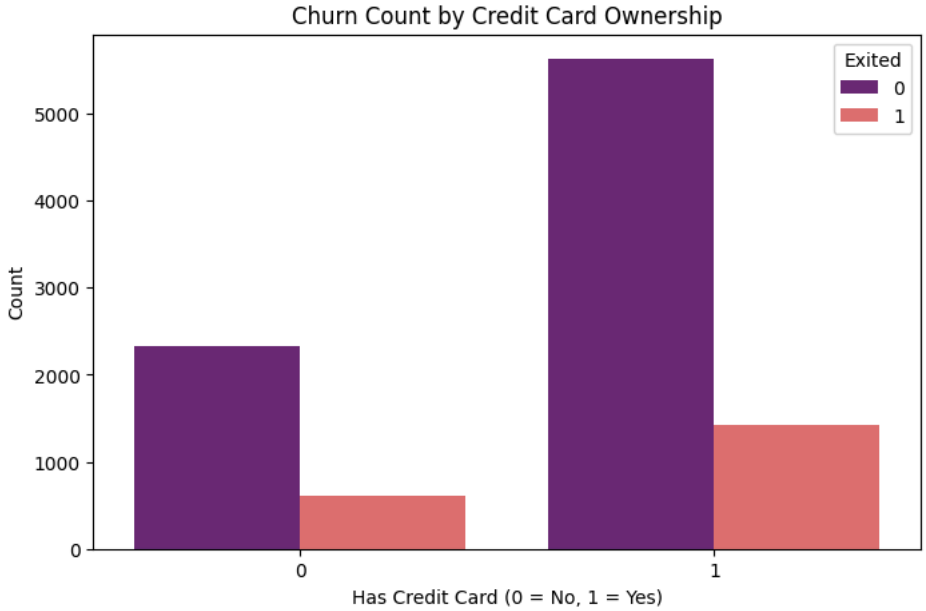


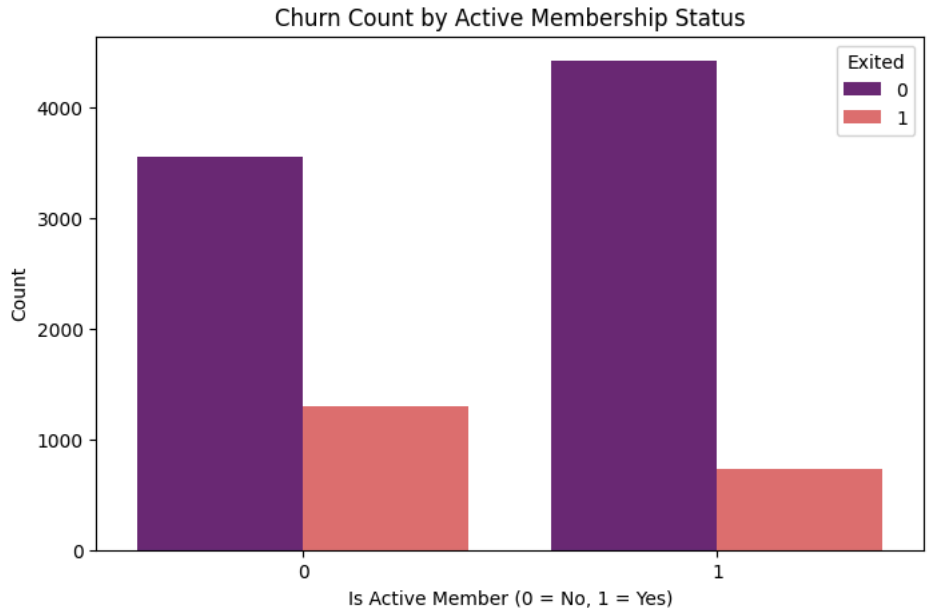
In [11]:

```
## Visualization of the distribution of the categorical features against the target feature

plt.figure(figsize=(8, 5))
sns.countplot(data=df, x="Geography", hue="Exited", palette="magma")
plt.title("Churn Count by Geography")
plt.xlabel("Geography")
plt.ylabel("Count")
plt.xticks(rotation=30)
plt.show()

plt.figure(figsize=(8, 5))
sns.countplot(data=df, x="Gender", hue="Exited", palette="magma")
plt.title("Churn Count by Gender")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.show()

plt.figure(figsize=(8, 5))
sns.countplot(data=df, x="HasCrCard", hue="Exited", palette="magma")
plt.title("Churn Count by Credit Card Ownership")
plt.xlabel("Has Credit Card (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.show()

plt.figure(figsize=(8, 5))
sns.countplot(data=df, x="IsActiveMember", hue="Exited", palette="magma")
plt.title("Churn Count by Active Membership Status")
plt.xlabel("Is Active Member (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.show()
```


Churn Count by Geography
Exited
0
1
Count
4000
3500
3000
2500
2000
1500
1000
500
0
France
Spain
Germany
Geography


Churn Count by Gender
Exited
0
1
Count
4000
3000
2000
1000
0
Female
Male
Gender

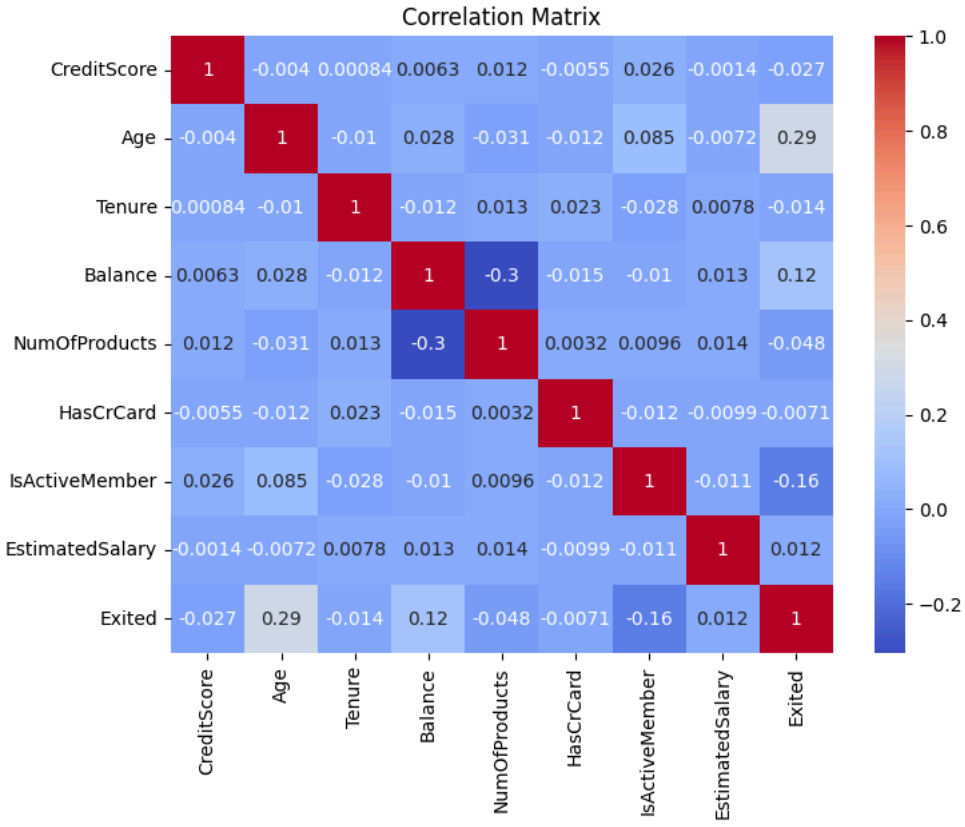

Churn Count by Credit Card Ownership
Exited
0
1
Count
5000
4000
3000
2000
1000
0
0
1
Has Credit Card (0 = No, 1 = Yes)



Churn rates do vary across the categorical features. Germany stands out with a noticeably higher churn rate compared to France and Spain. Gender does not seem to make much of a difference. Customers without a credit card churn slightly more, though the gap is not dramatic. The most notable finding here is that inactive members churn at a much higher rate, which suggests that engagement is one of the stronger indicators of whether a customer stays or leaves.

In [13]:

```
# This will allow us to identify features with strong correlation
# which will be helpful in understanding the relationship between the features

plt.figure(figsize=(8, 6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title('Correlation Matrix')
plt.show()
```



Most of the features show very weak correlations with each other, which is a good sign going into logistic regression since multicollinearity is not a major concern here. In terms of churn, Age has the most notable positive correlation at 0.29, so older customers are more likely to leave. IsActiveMember sits at -0.16, which lines up with what we saw earlier, inactive customers churn more. Balance has a slight positive correlation with churn at 0.12. The -0.3 between Balance and NumOfProducts is the strongest inter-feature correlation in the matrix, but it is not alarming. Overall the features are reasonably independent, which is what you want before fitting a logistic regression model.

In [14]:

```
# checking for missing values

df.isnull().sum()
```

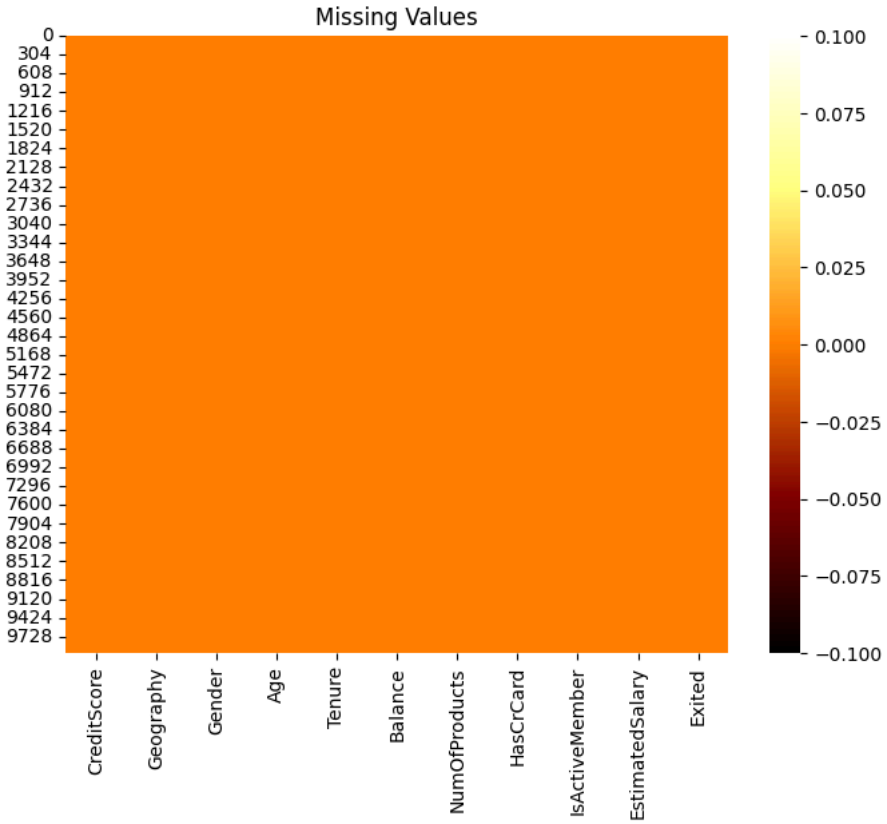
Out [14]:

```
CreditScore        0
Geography          0
Gender             0
Age                0
Tenure             0
Balance            0
NumOfProducts      0
HasCrCard          0
IsActiveMember     0
EstimatedSalary    0
```

```
Exited            0
dtype: int64
```

In [15]:

```
# Visualization of measing values using heatmap

plt.figure(figsize=(8, 6))
sns.heatmap(df.isnull(), cmap='afmhot')
plt.title('Missing Values')
plt.show()
```



The visualization confirms there are no missing values in the dataset. Since logistic regression requires numerical input, categorical variables like Geography and Gender need to be encoded before modeling. This is handled using one-hot encoding, which converts each category into a binary column the model can work with.

In [16]:

```
# Gender and Geography are categorical and need to be numerical before modeling.

# OneHotEncoder converts each category into binary columns, which are then

# merged back into the dataframe after dropping the original columns.

encoder = OneHotEncoder(sparse_output=False)
encoded = encoder.fit_transform(df[['Gender', 'Geography']])
encoded_df = pd.DataFrame(
    encoded,
    columns=encoder.get_feature_names_out(['Gender', 'Geography'])
)
df = df.drop(columns=['Gender', 'Geography'])
df = pd.concat([df, encoded_df], axis=1)
```

In [17]:

```
df.head()
```

Out [17]:

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited	Gender_Female	Gender_Male	Geography_France	Geog
0	619	42	2	0.00	1	1	1	101348.88	1	1.0	0.0	1.0	0.0
1	608	41	1	83807.86	1	0	1	112542.58	0	1.0	0.0	0.0	0.0
2	502	42	8	159660.80	3	1	0	113931.57	1	1.0	0.0	1.0	0.0
3	699	39	1	0.00	2	0	0	93826.63	0	1.0	0.0	1.0	0.0
4	850	43	2	125510.82	1	1	1	79084.10	0	1.0	0.0	0.0	0.0

LOGISTIC REGRESSION MODEL

In [18]:

```
# Split the dataset into features (x) and target (y), then divide into

# training and test sets using an 80/20 split with stratification to

# preserve the class distribution across both subsets.

x = df[['CreditScore', 'Age', 'Tenure', 'Balance', 'NumOfProducts',
      'IsActiveMember', 'EstimatedSalary', 'Gender_Female', 'Gender_Male',
     'Geography_France', 'Geography_Germany', 'Geography_Spain']]
y = df['Exited']

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42, stratify=y)
```

The dataset is split into features and the target variable, with Exited as what we are trying to predict. An 80/20 train-test split is used, meaning the model trains on 80% of the data and is evaluated on the remaining 20%. Stratification is applied to ensure both subsets maintain the same churn ratio, which is important given the class imbalance noted earlier.

In [29]:
```
# Train a Logistic Regression model on the training data.
# max_iter is set high to ensure the model converges properly.
# class_weight='balanced' is used to account for the class imbalance
# by giving more weight to the minority class (churners) during training.
model_df = LogisticRegression(max_iter=10000, class_weight='balanced')
model_df.fit(x_train, y_train)
```

Out [29]:
```
▾ LogisticRegression
▶ Parameters
```

In [30]:
```
# Predict on training data and preview the first 10 predictions
y_train_pred = model_df.predict(x_train)
print(y_train_pred[:10])
```

```
[1 1 1 0 1 0 0 0 0 0]
```

In [30]:
```
# Predict on training data and preview the first 10 predictions
y_train_pred = model_df.predict(x_train)
print(y_train_pred[:10])
```

```
[1 1 1 0 1 0 0 0 0 0]
```

In [31]:
```
# Compare actual vs predicted values side by side to get a quick
# visual check of how well the model is performing on the training set
res_df = pd.DataFrame(
    {'Actual': y_train,
     'Predicted': y_train_pred}
)
print(
    "Actual vs Predicted values:"
)
display(
    res_df.head(5)
)
```

```
Actual vs Predicted values:
```

	Actual	Predicted
2151	1	1
8392	1	1
5006	0	1
4117	0	0
7182	0	1

In [32]:
```
# Generate predictions on the test set and preview the first 10 results
y_test_pred = model_df.predict(x_test)
print(y_test_pred[:10])
```

```
[0 1 0 0 0 0 0 1 1 0]
```

In [33]:
```
# Compare actual vs predicted values on the test set
res_df = pd.DataFrame(
    {'Actual': y_test,
     'Predicted': y_test_pred}
)
display(res_df.head())
```

	Actual	Predicted
5702	0	0
3667	0	1
1617	0	0
5673	0	0
4272	0	0

In [34]:
```
# Extract model coefficients as a proxy for feature importance.
# Higher positive coefficients increase churn probability,
# while negative ones reduce it.
feat_imp = pd.DataFrame(
    {'Feature': x.columns,
     'Coefficient': model_df.coef_[0]}
)
feat_imp = feat_imp.sort_values(
    'Coefficient', ascending=False
)
display(feat_imp)
```

	Feature	Coefficient
10	Geography_Germany	1.535164e-01

	Feature	Coefficient
1	Age	7.596037e-02
3	Balance	2.716402e-06
6	EstimatedSalary	8.840736e-07
0	CreditScore	-9.435983e-04
2	Tenure	-6.653961e-03
4	NumOfProducts	-1.210154e-01
7	Gender_Female	-3.120681e-01
11	Geography_Spain	-6.349301e-01
9	Geography_France	-6.883688e-01
8	Gender_Male	-8.577148e-01
5	IsActiveMember	-8.817012e-01

In [35]:
```
# Evaluate model performance on the training set using accuracy score
y_train_pred = model_df.predict(x_train)
train_accuracy = accuracy_score(y_train, y_train_pred)
print(f"Training Accuracy: {train_accuracy:.4f}")
```

```
Training Accuracy: 0.7103
```

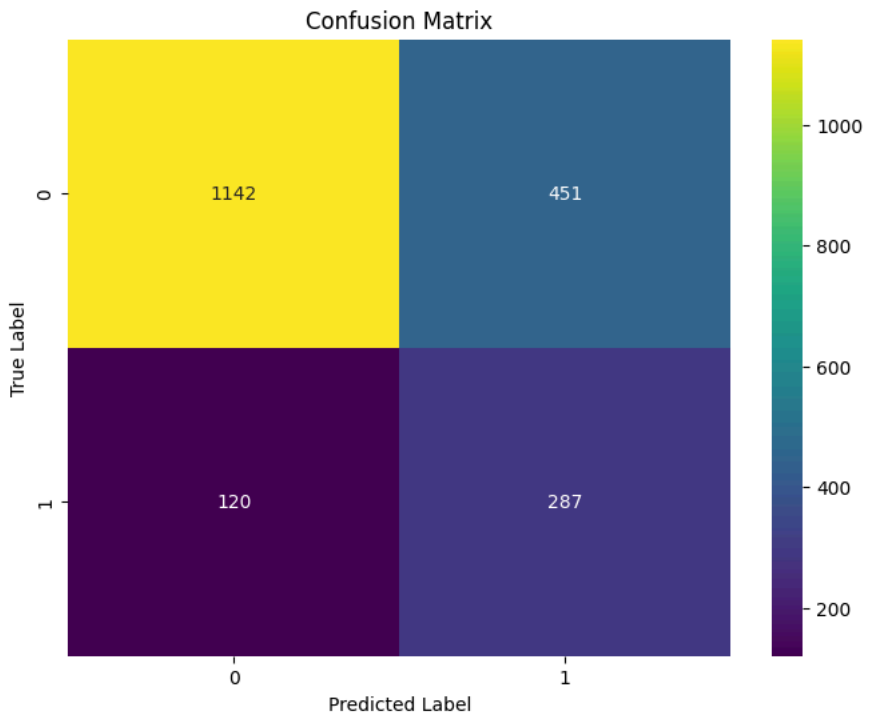
In [36]:
```
# Evaluate model performance on the test set using accuracy score
test_accuracy = accuracy_score(y_test, y_test_pred)
print(f"Testing Accuracy: {test_accuracy:.4f}")
```

```
Testing Accuracy: 0.7145
```

In [37]:
```
# Confusion matrix breaks down correct and incorrect predictions
# into true positives, true negatives, false positives, and false negatives
print(f"\nConfusion Matrix (Test Set):")
cm = confusion_matrix(y_test, y_test_pred)
print(cm)
```

```
Confusion Matrix (Test Set):
[[1142  451]
 [ 120  287]]
```

In [38]:
```
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='viridis')
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



The model was trained with class_weight='balanced' to account for the imbalance between churned and non-churned customers. Predictions were generated on both the training and test sets and checked against the actual labels as a quick sanity check. Feature importance from the model coefficients shows that Geography_Germany and Age have the strongest positive relationship with churn, while IsActiveMember is the strongest negative coefficient, confirming that active members are far less likely to leave. The model achieved a training accuracy of 71.03% and a test accuracy of 71.45%. While this is lower than a default model would produce, the confusion matrix tells a better story, with 287 churners correctly identified and false negatives reduced significantly. For a churn prediction task, catching actual churners matters more than raw accuracy, and the balanced model reflects that priority.

DECISION TREE REGRESSOR

In [39]:
```
# The same feature set and target variable are reused from the logistic regression.
# The 80/20 stratified split is kept consistent to ensure fair comparison
# across all three models.
```

```
x = df[['CreditScore', 'Age', 'Tenure', 'Balance', 'NumOfProducts',
      'IsActiveMember', 'EstimatedSalary', 'Gender_Female', 'Gender_Male',
     'Geography_France', 'Geography_Germany', 'Geography_Spain']]
y = df['Exited']
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42, stratify=y)
```

In [40]:

```
# Train a Decision Tree Regressor with controlled depth and leaf size
# to prevent overfitting on the training data.
df_regressor = DecisionTreeRegressor(
    max_depth=10,
    min_samples_split=20,
    min_samples_leaf=10,
    random_state=42
)
df_regressor.fit(x_train, y_train)
```

Out [40]:

```
▾ DecisionTreeRegressor
▶ Parameters
```

A Decision Tree Regressor is trained on the data. max_depth limits how deep the tree grows, min_samples_split controls the minimum samples needed to split a node, and min_samples_leaf ensures each leaf has enough data points. These constraints work together to prevent the model from memorizing the training data.

In [41]:

```
# Generate continuous predictions on the training set
y_pred_train = df_regressor.predict(x_train)
print(y_pred_train)
```

```
[0.91304348 0.17391304 0.21621622 ... 0.03448276 0.         0.61290323]
```

Unlike the logistic regression which outputs clean 0s and 1s, the Decision Tree Regressor outputs continuous values between 0 and 1, representing the proportion of churners in each leaf node. These will need to be converted to binary labels before evaluation.

In [42]:

```
# Generate continuous predictions on the test set
y_pred_test = df_regressor.predict(x_test)
print(y_pred_test)
```

```
[0.         0.09090909 0.         ... 0.9        0.         0.07739308]
```

The same continuous predictions are now made on the unseen test data. This is the more critical set since the model had no exposure to it during training.

In [43]:

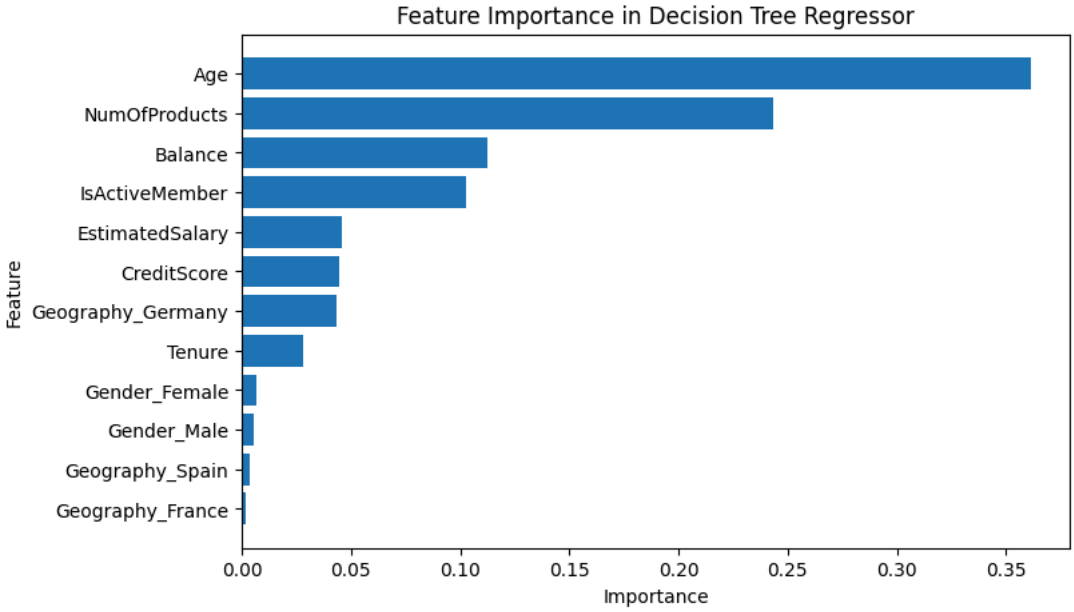
```
# Extract and rank feature importance scores from the Decision Tree
feature_importance = pd.DataFrame({
    'feature': x.columns,
    'importance': df_regressor.feature_importances_
}).sort_values('importance', ascending=False)

display(feature_importance)
```

	feature	importance
1	Age	0.361111
4	NumOfProducts	0.243426
3	Balance	0.112226
5	IsActiveMember	0.102866
6	EstimatedSalary	0.045739
0	CreditScore	0.044779
10	Geography_Germany	0.043649
2	Tenure	0.028389
7	Gender_Female	0.006579
8	Gender_Male	0.005746
11	Geography_Spain	0.003685
9	Geography_France	0.001804

Unlike logistic regression which uses coefficients, the Decision Tree measures feature importance based on how much each feature reduces impurity across all splits. Features ranked higher are relied on more heavily when the tree makes its decisions.

In [44]:

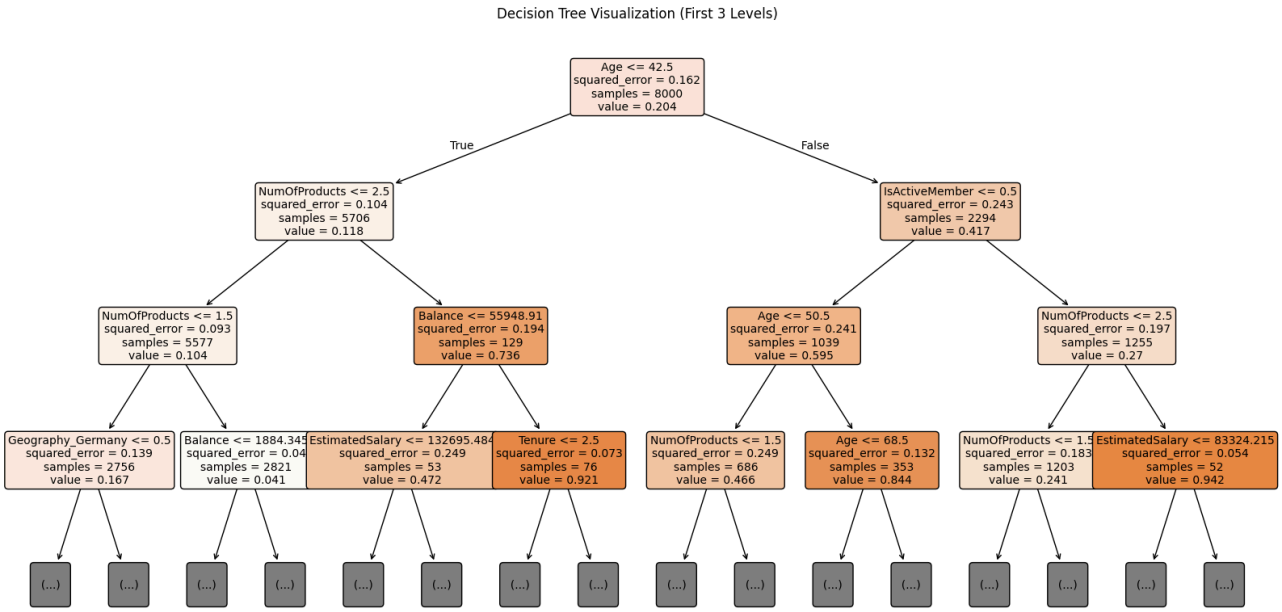
```
# Visualize feature importance as a horizontal bar chart
sorted_importance = feature_importance.sort_values(by='importance', ascending=True)

plt.figure(figsize=(8, 5))
plt.barh(sorted_importance['feature'], sorted_importance['importance'])
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.title('Feature Importance in Decision Tree Regressor')
plt.show()
```



Age, NumOfProducts and Balance come out as the most important features, which is consistent with what was observed during EDA and in the logistic regression. IsActiveMember also remains a notable contributor.

In [45]:

```
# Visualize the first 3 levels of the Decision Tree for readability
plt.figure(figsize=(20, 10))
features = x.columns
tree.plot_tree(df_regressor,
               feature_names=features,
               max_depth=3,
               filled=True,
               rounded=True,
               fontsize=10)
plt.title('Decision Tree Visualization (First 3 Levels)')
plt.show()
```



Visualizing the top 3 levels of the tree shows the most critical decision splits. The root node reveals which feature the model considers most informative for separating churners from non-churners at the very first split.

In [64]:

```
# Convert continuous predictions to binary class labels.
# A threshold of 0.4 is used instead of the default 0.5 to improve
# sensitivity toward the minority class (churners).
y_pred_train_class = (y_pred_train > 0.4).astype(int)
y_pred_test_class = (y_pred_test > 0.4).astype(int)
```

Since the regressor outputs probabilities, a threshold is needed to convert them into binary predictions. Lowering the threshold from 0.5 to 0.4 makes the model more aggressive in flagging potential churners, which helps address the class imbalance without needing extra libraries.

In [65]:

```
# Evaluate Decision Tree performance on both sets after threshold conversion
train_accuracy = accuracy_score(y_train, y_pred_train_class)
test_accuracy = accuracy_score(y_test, y_pred_test_class)

print("Decision Tree Training Accuracy:", train_accuracy)
print("Decision Tree Testing Accuracy:", test_accuracy)
```

```
Decision Tree Training Accuracy: 0.87925
Decision Tree Testing Accuracy: 0.8295
```

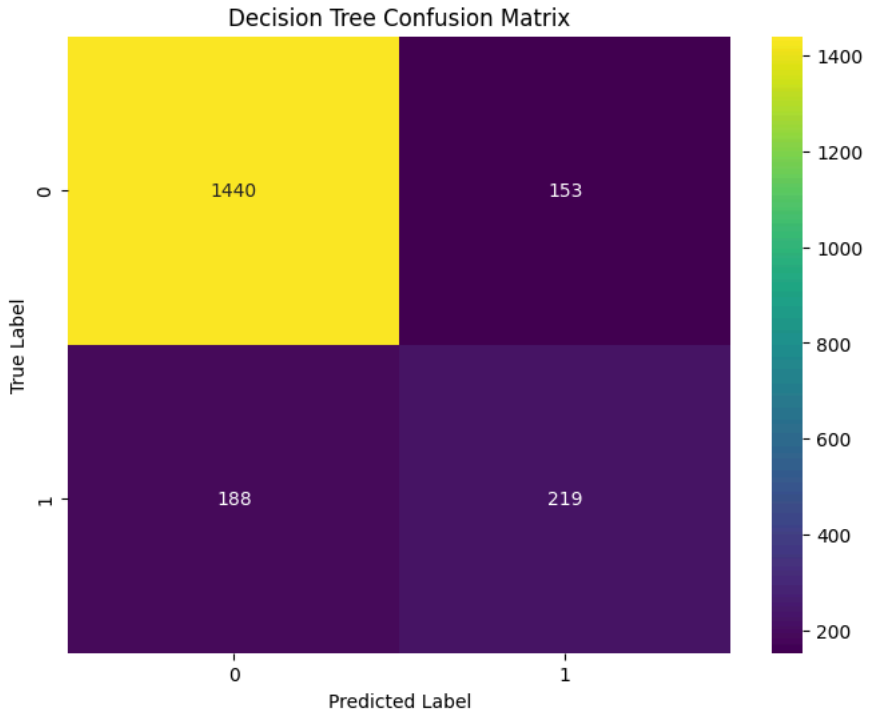
In [66]:

```
# Confusion matrix to evaluate classification performance on the test set
cm_tree = confusion_matrix(y_test, y_pred_test_class)
print("Confusion Matrix:")
print(cm_tree)
```

```
Confusion Matrix:
[[1440  153]
 [ 188  219]]
```

In [67]:

```
# Visualize the confusion matrix as a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(cm_tree, annot=True, fmt='d', cmap='viridis')
plt.title('Decision Tree Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



The Decision Tree hits a training accuracy of 87.93% and test accuracy of 82.95%, which is a decent result with no major overfitting. The confusion matrix tells the more interesting story though. With the threshold lowered to 0.4, the model caught 219 churners compared to just 77 from the original logistic regression, and false negatives dropped to 188. There is still room to improve, but this is a clear step in the right direction heading into the Random Forest.

RANDOM FOREST REGRESSOR

In [68]:

```
# Train a Random Forest Regressor using 100 decision trees.
# The same constraints from the Decision Tree are applied to each tree.
# n_jobs=-1 uses all available CPU cores to speed up training.
rf_regressor = RandomForestRegressor(
    n_estimators=100,
    max_depth=10,
    min_samples_split=20,
    min_samples_leaf=10,
    random_state=42,
    n_jobs=-1
)
rf_regressor.fit(x_train, y_train)
```

Out [68]:

```
▾ RandomForestRegressor
▶ Parameters
```

In [69]:

```
# Generate averaged probability predictions from all 100 trees
y_pred_train_rf = rf_regressor.predict(x_train)
y_pred_test_rf = rf_regressor.predict(x_test)

print("Random Forest Predictions (Training Set - first 10):")
print(y_pred_train_rf[:10])
print("\nRandom Forest Predictions (Test Set - first 10):")
print(y_pred_test_rf[:10])
```

```
Random Forest Predictions (Training Set - first 10):
[0.95319726 0.33321363 0.16969392 0.00970652 0.24130009 0.02262516
 0.37452713 0.05394879 0.09375087 0.09850711]

Random Forest Predictions (Test Set - first 10):
[0.00812969 0.04079632 0.01252313 0.02852934 0.09553995 0.27541332
 0.02827324 0.0586081  0.59103771 0.10376693]
```

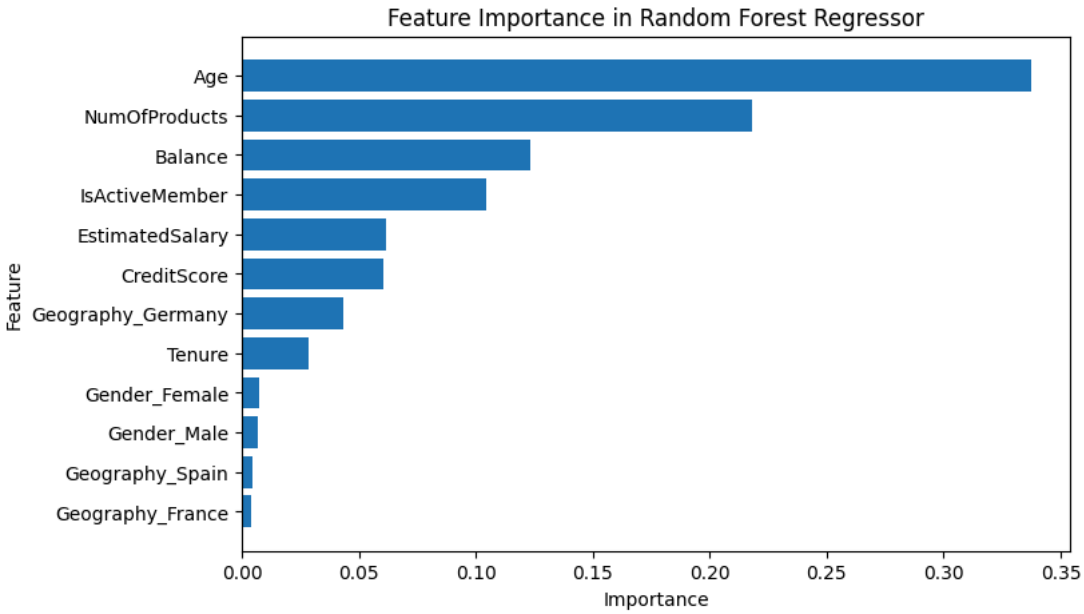
The Random Forest averages the predictions across all 100 trees, producing smoother and more stable probability estimates compared to a single Decision Tree.

In [70]:
```
# Extract feature importance averaged across all 100 trees
rf_feature_importance = pd.DataFrame({
    'feature': x.columns,
    'importance': rf_regressor.feature_importances_
}).sort_values('importance', ascending=False)

display(rf_feature_importance)
```

	feature	importance
1	Age	0.337126
4	NumOfProducts	0.218276
3	Balance	0.123276
5	IsActiveMember	0.104592
6	EstimatedSalary	0.061473
0	CreditScore	0.060463
10	Geography_Germany	0.043106
2	Tenure	0.028553
7	Gender_Female	0.007417
8	Gender_Male	0.007095
11	Geography_Spain	0.004374
9	Geography_France	0.004247

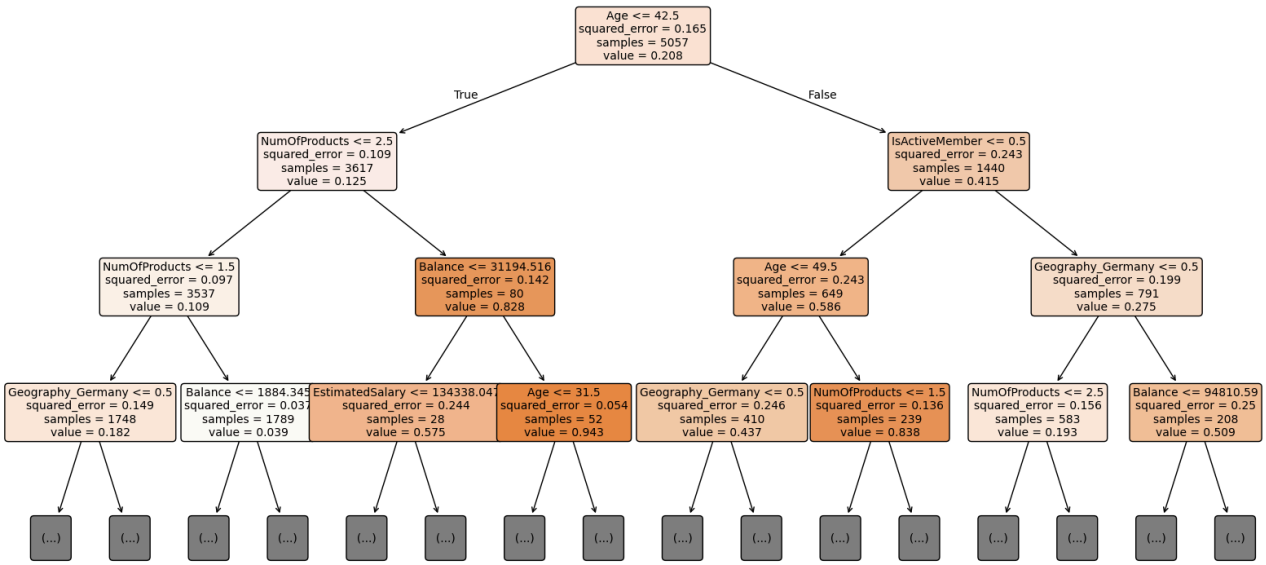
Averaging importance scores across 100 trees makes this ranking more robust than a single tree. The most influential features remain consistent with earlier models, lending confidence to those findings.

In [71]:
```
# Visualize Random Forest feature importance
sorted_importance_rf = rf_feature_importance.sort_values(by='importance', ascending=True)

plt.figure(figsize=(8, 5))
plt.barh(sorted_importance_rf['feature'], sorted_importance_rf['importance'])
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.title('Feature Importance in Random Forest Regressor')
plt.show()
```



In [72]:
```
# Visualize the first tree in the Random Forest (first 3 levels only)
plt.figure(figsize=(20, 10))
features = x.columns
tree.plot_tree(rf_regressor.estimators_[0],
               feature_names=features,
               max_depth=3,
               filled=True,
               rounded=True,
               fontsize=10)
plt.title('Random Forest - Visualization of the First Tree (First 3 Levels)')
plt.show()
```

Random Forest - Visualization of the First Tree (First 3 Levels)



Since the Random Forest is made up of 100 trees, only the first tree is visualized here for reference. It gives a rough idea of the structure each individual tree follows, though the full model's strength comes from combining all of them together.

In [83]:
```
# Convert averaged probabilities to binary class labels using a 0.4 threshold
y_pred_train_rf_class = (y_pred_train_rf > 0.3).astype(int)
y_pred_test_rf_class = (y_pred_test_rf > 0.3).astype(int)
```

In [84]:
```
# Evaluate Random Forest performance on both sets
rf_train_accuracy = accuracy_score(y_train, y_pred_train_rf_class)
rf_test_accuracy = accuracy_score(y_test, y_pred_test_rf_class)

print("Random Forest Training Accuracy:", rf_train_accuracy)
print("Random Forest Testing Accuracy:", rf_test_accuracy)
```

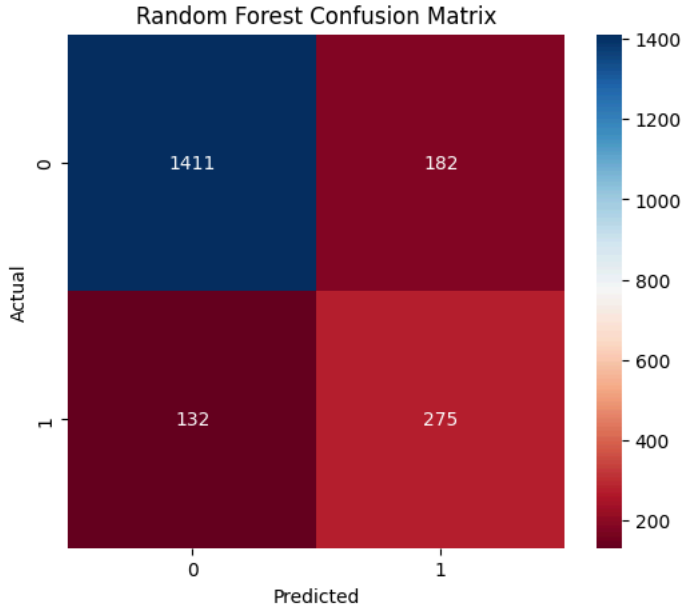
```
Random Forest Training Accuracy: 0.8815
Random Forest Testing Accuracy: 0.843
```

In [85]:
```
# Confusion matrix for the Random Forest on the test set
cm_rf = confusion_matrix(y_test, y_pred_test_rf_class)
print("Random Forest Confusion Matrix:")
print(cm_rf)
```

```
Random Forest Confusion Matrix:
[[1411  182]
 [ 132  275]]
```

The Random Forest uses a threshold of 0.3 rather than 0.4 because while 0.4 gives a higher overall accuracy, 0.3 catches 275 churners compared to 225, with only 132 missed. In a churn prediction task, catching more at-risk customers matters more than raw accuracy, making 0.3 the better practical choice. This also makes the Random Forest the strongest performing model in the project overall.

In [88]:
```
# Visualize the Random Forest confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='RdBu')
plt.title('Random Forest Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```


Random Forest Confusion Matrix
1411
182
132
275
Actual
0
1
Predicted
0
1
1400
1200
1000
800
600
400
200

In [ ]: